05_layer_normalization

Visual guide — Layer Normalization

This note complements the [official Triton tutorial](https://triton-lang.org/main/getting-started/tutorials/05-layer-norm.html). Read it before diving into code.

Goal

Fused layer norm over the last dimension with one pass per row/group.

Mental model

```mermaid
flowchart TB
  R[load row] --> MU[mean / var]
  MU --> N[normalize]
  W[weight / bias] --> N
  N --> Wout[store]
```

Official benchmark figure



Layer norm performance comparison.

Step-by-step (what each phase does)

1. **Host setup** — Allocate tensors on `DEVICE`, pick dtypes, and define launch grid meta-parameters (`BLOCK_SIZE`, tile sizes, `num_warps`).
2. **Kernel** — Each program instance loads a tile, computes in fast memory (SRAM/shared), stores results. Use **masks** when dimensions are not multiples of the tile.
3. **Correctness** — Compare to PyTorch / reference with `torch.allclose` on odd sizes (tests padding paths).
4. **Benchmark** — Use `triton.testing.do_bench` and `perf_report` for GB/s or TFLOPS; interpret as memory-bound vs compute-bound.

Tips

- If performance flatlines, check occupancy (registers/shared mem) and whether the kernel is **memory-bound** (fusion helps) or **compute-bound** (tile sizes / autotune matter).
- Re-run `python scripts/download_tutorial_images.py` if figures are missing offline.

See also

- [Tutorials index](https://triton-lang.org/main/getting-started/tutorials/index.html)
- Repo notebook: `05_layer_normalization.ipynb`

benchmark-layer-norm.png

